Name:Priyaan G

Srn: PES2UG24CS376

Task1 Handel inputs for helicopter nad set proper bounds:

From helicopter.py

```
    def handle_input(self, keys_pressed):
        if keys_pressed[pygame.K_UP]:
            if self.vy > -3 * THRUST:
                self.vy -= THRUST
        if keys_pressed[pygame.K_DOWN]:
            if self.vy < 3 * THRUST:
                self.vy += THRUST

    def update(self, height_bound):
        self.y += self.vy
        if self.y - self.height / 2 < 0:
            self.y = self.height / 2
            self.vy = 0
        if self.y + self.height / 2 > height_bound:
            self.y = height_bound - self.height / 2
            self.vy = 0
```

Task2 obstacle collision game end:

From game_engine.py

```
def update(self):
    self.helicopter.update(HEIGHT)

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_obstacle()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obstacle in self.obstacles:
        obstacle.update()

    self.obstacles = [o for o in self.obstacles if not o.is_off_screen()]

    # Precise Collision Detection using Rects
    heli_rect = self.helicopter.get_rect()
    for obstacle in list(self.obstacles):
        if heli_rect.colliderect(obstacle.get_top_rect()) or heli_rect.colliderect(obstacle.get_bottom_rect()):
            if self.shield_active:
                # Shield absorbs the hit and deactivates
                self.shield_active = False
                self.last_shield_used_time = time.time()
                self.obstacles.remove(obstacle)
            else:
                return False

    return True
```

```
def hope(self, surface, font):
    from game import renderer
    ftime = int(time.time())
    score = (ftime - self.itime) * SCROLL_SPEED
    renderer.draw_banner(surface, font, f"Points: {score}")
```

Task3 points implementation and game restart:

From game_engine.py:

```
def hope(self, surface, font):
    from game import renderer
    ftime = int(time.time())
    score = (ftime - self.itime) * SCROLL_SPEED
    renderer.draw_banner(surface, font, f"Points: {score}")
```

From main.py:

```
running = True
while running:
    # Initialize a fresh engine for each game session
    engine = GameEngine()
    game_over = False

    # --- Main Gameplay Loop ---
    while running and not game_over:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)

        # engine.update() returns False when the game terminates
        if engine.update() is False:
            game_over = True

        engine.draw(screen, font)
        pygame.display.flip()
        clock.tick(60)

    # --- Game Over / Wait for Key Press Loop ---
    if running:
        engine.hope(screen, font)
        pygame.display.flip()

        waiting_for_key = True
        while running and waiting_for_key:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False
                    waiting_for_key = False
                elif event.type == pygame.KEYDOWN:
                    # User pressed any key, restart game
                    waiting_for_key = False

            clock.tick(60)

pygame.quit()
```

Task4 implement Shield

From render.py

```
def draw_scene(surface, helicopter, obstacles, shield_active, shield_cooldown_left, font):
    surface.fill(COLOR_BG)
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_bottom_rect())

    # Draw Shield Aura if active
    if shield_active:
        shield_rect = helicopter.get_rect().inflate(10, 10)
        pygame.draw.rect(surface, COLOR_SHIELD, shield_rect, width=3, border_radius=6)

    pygame.draw.rect(surface, COLOR_HELI, helicopter.get_rect(), border_radius=4)

    # Render UI Text for Shield Status
    if shield_active:
        status_text = "Shield: ACTIVE (Press H)"
        color = (0, 100, 200)
    elif shield_cooldown_left > 0:
        status_text = f"Shield Cooldown: {int(shield_cooldown_left)}s"
        color = (120, 120, 120)
    else:
        status_text = "Shield: READY (Press H)"
        color = (0, 140, 0)

    draw_text(surface, font, status_text, (20, 20), color)
```

From game_engine.py

```
def handle_keydown(self, key):
    if key == pygame.K_h:
        current_time = time.time()
        # Check if shield is not active and cooldown has passed
        if not self.shield_active and (current_time - self.last_shield_used_time >= SHIELD_COOLDOWN):
            self.shield_active = True
```